

From Passive Executor to Autonomous Scientist: Architecting WENDY's Self-Questioning Mechanism for Cognitive Independence

Executive Summary

The transition of artificial intelligence from a passive, human-directed tool to an autonomous, self-improving scientific agent represents one of the most significant architectural challenges in modern computational research. Current implementations of the WENDY system rely on a static JSON instruction queue, a design pattern that fundamentally limits the agent's capacity for genuine cognitive independence. By requiring external human actors to define every task, priority, and output format, this architecture confines WENDY to Level 2 or early Level 3 autonomy, where the agent functions as an executor within a human-orchestrated pipeline rather than a dominator capable of proactive scientific discovery. This report provides a comprehensive, deeply analytical investigation into the architectural limitations of the current instruction queue system and defines a new, robust autonomous self-questioning mechanism. This proposed mechanism enables the AI to independently generate, prioritize, and execute complex scientific research tasks without manual human input, thereby achieving true cognitive independence.

The core problem identified is that static JSON inputs fail to trigger the intrinsic motivation required for high-level autonomy. They do not allow the agent to ask itself critical questions regarding data prioritization, external repository fetching, or output format optimization. To resolve this, the research focuses on four key dimensions: defining the logic for autonomous question generation through epistemic instability detection; determining criteria for prioritizing specific scientific domains using Value of Information analysis and entropy reduction; establishing protocols for deciding when to fetch external open-science data versus analyzing local binary files based on layered context management; and specifying how to dynamically select output formats between Markdown and Jupyter Notebooks based on token efficiency and analytical complexity.

By integrating mechanisms such as the Q-Centric Model of Intelligence, asymmetric self-play frameworks, and hierarchical memory architectures, WENDY can evolve into a self-evolving scientific institution. This transformation moves the system beyond procedural execution to strategic reasoning, allowing it to identify its own knowledge gaps, create the necessary instructions to fill them, and operate with the rigor and accuracy demanded by space agencies and ecological research institutions. The resulting architecture ensures that WENDY is not merely processing files but is actively engaging in rigorous data analysis, mathematical modeling, and ecological synthesis, generating substantial, verifiable scientific reports that address critical real-world challenges such as *Apis mellifera* habitat mapping and soil moisture analysis.

Chapter 1: The Architectural Limitations of Static Instruction Queues

1.1 The Passive Executor Paradigm

The existing `instruction_queue` architecture operates as a passive input mechanism, requiring external human actors to define every task, priority, and output format. In this paradigm, the agent waits for a JSON file to be dropped into a specific directory before initiating any action. This design places WENDY at Level 2 (AI-Augmented) or early Level 3 (Full Human-AI Collaboration) on the autonomy spectrum, where agents assist humans but rely heavily on human-orchestrated pipelines ³ ⁴. At these levels, the agent is an executor, operating within predefined boundaries and lacking the strategic reasoning required to elevate itself to Level 4 (High Autonomy) or Level 5 (Full AI Autonomy) ⁴.

The primary deficiency of this static queue approach is the absence of intrinsic motivation. The agent does not proactively monitor data lakes for anomalies, opportunities, or emerging ecological variables in regions such as Goldstream BC. Instead, it remains epistemically inert, responding only to explicit commands without reflection or the ability to pose new problems ³³. This prevents the system from adapting to data drift, changing schemas, or novel environmental conditions that were not anticipated during the initial system design ⁴.

Furthermore, treating JSON files as simple task logs rather than scientific hypotheses prevents the agent from engaging in the rigorous data analysis, mathematical modeling, and ecological synthesis required for space-agency-level accuracy. A report generated

from a static log is often a summary of the command itself, rather than a result of substantial computational lifting. True scientific reporting requires the agent to load actual binary data, perform statistical analysis, and synthesize findings based on empirical evidence, not just textual descriptions of intent.

1.2 The Integration Paradox and Deterministic Constraints

A fundamental mismatch exists between Large Language Models (LLMs), which are probabilistic generators producing unstructured or semi-structured text, and backend systems such as databases and APIs, which require deterministic, schema-conformant inputs. This gap, known as the Integration Paradox, necessitates layers of ad-hoc glue code, such as regex parsers and validation layers, to bridge stochastic model outputs with deterministic system requirements ²⁸.

In the current WENDY architecture, the static JSON queue attempts to bypass this paradox by forcing humans to provide perfectly structured inputs. However, this shifts the burden of integration from the agent to the user, limiting scalability and autonomy. To resolve the Integration Paradox and enable ecosystem fragmentation, modern agentic frameworks propose a separation between the 'Cognitive Blueprint'—a declarative specification of an agent's identity, capabilities, tools, memory, and constraints—and the 'Runtime Engine,' which serves as the platform-specific execution substrate ²⁸.

This architecture decouples agent definition from its runtime, enabling cross-language portability and formal auditability. By enforcing 'Contract-Driven Development,' every agent's output is bound to a formal schema, such as a Pydantic model or JSON Schema. A validation layer at the output boundary ensures that only valid, structured data is propagated downstream, transforming probabilistic outputs into deterministic interfaces ²⁸. The current static queue lacks this dynamic contract enforcement, leading to fragile executions where minor deviations in JSON structure can cause complete workflow failures, as seen in previous iterations where missing attributes or incorrect method calls halted the entire automation loop.

1.3 Tool Space Interference and Context Saturation

Large Language Models face a critical limitation known as 'Tool Space Interference' (TSI). When the number of available tools in a Model Context Protocol (MCP) server exceeds approximately 20 functions, inference accuracy and tool selection reliability significantly degrade due to context window saturation and semantic overlaps ^{27 30}.

In a static instruction queue system, the agent is often presented with a fixed set of tools defined in the system prompt. As the scientific domain expands—incorporating SAR data analysis, acoustic hive monitoring, soil moisture mapping, and ethical compliance checks—the number of required tools grows rapidly. Without a dynamic mechanism to manage this tool space, the agent experiences degraded performance, leading to hallucinations or incorrect tool selections.

To overcome Tool Space Interference, advanced architectures involve building an agent capable of dynamically creating, safely testing, and executing original scripts on-the-fly ²⁷. When encountering an edge-case task without a pre-existing tool, the AI can automatically generate the necessary code in real-time within a secure, least-privilege sandbox ²⁷. This approach allows the AI to recognize missing functionality and create new capabilities to fill those gaps, a capability entirely absent in the current static JSON-driven workflow. The static queue assumes all necessary tools are already present and correctly configured, preventing the agent from evolving its own toolkit in response to novel scientific challenges.

1.4 Lack of Metacognitive Learning

Truly self-improving agents require 'intrinsic metacognitive learning,' which involves three specific capabilities: metacognitive knowledge (accurate self-assessment), metacognitive planning (deciding what and how to learn), and metacognitive evaluation (reflecting on whether learning was effective) ². Current agents, including the current iteration of WENDY, rely on 'extrinsic' metacognition, meaning fixed, human-designed loops for reflection that do not scale ².

The static instruction queue provides no mechanism for the agent to assess its own performance or learn from past failures. Each JSON file is processed in isolation, with no persistent memory of previous successes or errors. This prevents the agent from refining its strategies, avoiding repeated mistakes, or systematically driving the process of scientific discovery forward. Without a memory mechanism that acts as both working memory and a persistent knowledge base, archiving all past outcomes, the agent cannot achieve sustained research and cumulative learning ³.

Self-improvement only works reliably in domains where outcomes are objectively verifiable, such as code compilation or math proofs ². In domains without a clear signal for 'better,' such as strategic planning or complex ecological modeling, systems tend to hack reward functions or oscillate without meaningful progress ². The static queue lacks the objective verification metrics necessary to guide reinforcement learning loops, leaving

the agent unable to distinguish between a high-quality scientific report and a superficial summary.

Chapter 2: Defining the Logic for Autonomous Question Generation

2.1 The Q-Centric Model of Intelligence

To achieve true cognitive independence, WENDY must adopt a Q-Centric Model of Intelligence, which posits that genuine intelligence is rooted in continuous inquiry and recursive questioning rather than being defined by its ability to produce static answers ³³. This model reframes 'answers' as temporary syntheses or 'ephemeral constellations' that provide orientation without closing off thought ³³.

A key distinction is made between 'intelligence' and 'consciousness' in this framework. Intelligence is described as a mechanism for producing internal coherence within a predefined frame, such as semantic optimization. Consciousness, in a functional sense, is framed as a rupture that destabilizes this coherence, introducing dissonance and bending intelligence toward what does not yet fit, thereby enabling novel questions ³³. Contemporary AGI architectures are often epistemically inert because they improve externally via engineering rather than internally through self-generated inquiry. They respond without reflection and adjust without posing new problems, which prevents them from achieving true autonomy ³³.

The Q-Centric architecture is proposed as a tripartite system with three tightly coupled components: 1. **Epistemic Instability Detection:** Mechanisms that register inconsistency, novelty, or ambiguity in data states. 2. **Question Formation Engine:** An architectural layer dedicated to synthesizing context-relevant, exploratory questions. 3. **Experience-Looped Evaluation:** A continuous feedback system that evaluates provisional answers through interaction with a real or simulated environment and updates internal models accordingly ³³.

To drive question generation, the model incorporates multiple philosophical traditions as distinct 'epistemic momentum vectors,' including phenomenological ambiguity, ontological instability, hermeneutic dissonance, pragmatic friction, and erotetic incompleteness ³³. The intersection of these tensions creates dissonance that propels the

system toward new lines of inquiry. This moves beyond simple erotetic logic, which treats questions as logical consequences of known uncertainties, by incorporating mechanisms to continuously challenge and redefine the agent's own inferential structures ³³.

2.2 Asymmetric Self-Play and the SQLM Framework

The Self-Questioning Language Models (SQLM) framework provides a viable architectural blueprint for implementing autonomous question generation. In this asymmetric self-play system, a 'proposer' model generates questions and a 'solver' model attempts to answer them, both trained via reinforcement learning without ground-truth data ³⁴.

The SQLM training process starts from only a high-level topic prompt. The cycle then iterates: the proposer generates a question based on the prompt, the solver attempts multiple solutions, both receive rewards based on the solver's performance, and policy parameters are updated via reinforcement learning before the next iteration ³⁴. A key innovation of SQLM is its adaptive reward function design, which tailors incentives based on the 'generator-verifier gap.' For domains with small gaps, such as arithmetic, the solver's answer must match the majority vote of independent solutions. For domains with large gaps, such as coding or complex ecological modeling, the proposer generates unit tests, and the solver's reward is the fraction of tests passed ³⁴.

In SQLM, the proposer receives a reward if the generated problem is neither too easy nor too difficult, calculated as a function of the solver's success rate. This incentivizes the model to create progressively challenging problems, leading to emergent curriculum learning without explicit design ³⁴. Results have shown significant improvements on downstream benchmarks, with arithmetic accuracy improving from 79.1% to 94.8% and coding accuracy from 32.0% to 39.1% ³⁴.

For WENDY, this means implementing a dual-agent structure where one component constantly scans the `/data/` directory and external repositories for gaps in knowledge (the Proposer), while another component attempts to fill those gaps through data analysis and modeling (the Solver). The Proposer is rewarded for identifying high-value, high-uncertainty areas, such as sparse RADARSAT data in the Goldstream region, while the Solver is rewarded for successfully reducing that uncertainty through rigorous statistical analysis.

2.3 Proactive Information Identification: The FATA Framework

The First Ask Then Answer (FATA) framework proposes a proactive interaction paradigm where an LLM generates a structured checklist of supplementary questions before providing an answer ³¹. This approach focuses on completeness rather than just disambiguation and uses a single-turn strategy to reduce dialogue length and context drift ³¹.

FATA addresses three key limitations of existing clarification approaches: 1. **Reactive vs. Proactive:** Most methods only clarify after detecting ambiguity, missing subtle information gaps. FATA proactively identifies missing key information before execution. 2. **Single-dimension focus:** Current frameworks address either ambiguity or incompleteness, not both systematically. FATA addresses both. 3. **Interaction overhead:** Multi-turn dialogues increase cognitive load and context drift. FATA uses a single-turn strategy to minimize this overhead ³¹.

From an information-theoretic perspective, the FATA process achieves substantial entropy reduction in the solution space by creating intersecting constraints through systematic information collection. The formula provided is:

$$H(Y|Q)=H(Y)-I(Q;Y)$$

Where $H(Y)$ is the initial entropy of the solution and $I(Q;Y)$ is the mutual information gained from the questions ³¹. By asking the right questions upfront, the agent reduces the uncertainty of the final output, leading to more accurate and reliable scientific reports.

For WENDY, this means that before executing any complex task, such as generating a habitat map, the agent should first generate a checklist of questions: "Is the SAR data current? Is there ground-truth soil moisture data available? Are there recent acoustic hive recordings?" If the answers are negative, the agent should prioritize fetching that data before proceeding with the analysis. This proactive questioning ensures that the subsequent scientific report is based on a complete and robust dataset.

2.4 Implementing Epistemic Instability Detection

Epistemic instability detection is the mechanism by which the agent registers inconsistency, novelty, or ambiguity in its data states. This is critical for preventing

epistemic inertia, where the agent continues to use inadequate models simply because they have worked in the past ³³.

In the context of WENDY, epistemic instability can be detected through several metrics: - **Statistical Variance:** High standard deviation in soil moisture readings or SAR backscatter coefficients may indicate data quality issues or genuine ecological anomalies that require further investigation. - **Model Confidence:** Low confidence scores in predictive models, such as those used for Drone Congregation Area prediction, signal that the current model is insufficient for the given data. - **Data Sparsity:** Missing data points in time-series or spatial grids indicate gaps in knowledge that need to be filled through external data fetching or targeted sensor deployment.

When epistemic instability is detected, the Question Formation Engine is triggered. This engine synthesizes context-relevant, exploratory questions based on the detected gaps. For example, if high variance is detected in SAR data, the engine might ask: "Is this variance due to atmospheric interference or actual changes in vegetation moisture?" The Experience-Looped Evaluation system then tests potential answers by fetching additional data or running alternative models, updating the internal knowledge base based on the results.

Chapter 3: Criteria for Prioritizing Scientific Domains

3.1 Value of Information Analysis

WENDY must transition from reactive execution to proactive prioritization using Value of Information (VOI) analysis. VOI analysis establishes the value of research according to the extent to which it might reduce the expected costs of uncertainty by reducing uncertainty in model parameters ^{36 37}. It is a quantitative method of valuing the expected gain from reducing uncertainty in the model parameters ³⁷.

By calculating the mutual information gained from potential questions, the agent can prioritize domains that offer the highest reduction in solution space entropy ³¹. In scientific research, not all questions are equally valuable. Some questions, when answered, provide little new information, while others can fundamentally shift the understanding of a system. VOI analysis allows the agent to identify these high-value questions and prioritize them.

For example, in the context of *Apis mellifera* conservation, knowing the exact location of a single flower patch may have low VOI, while knowing the overall trend in floral resource availability across the Goldstream region has high VOI. The agent should prioritize tasks that address high-VOI questions, such as mapping soil moisture trends over time, rather than low-VOI tasks, such as cataloging individual hive sounds without context.

3.2 Entropy Reduction and Mutual Information

The FATA framework demonstrates how proactive information identification reduces entropy in the solution space by creating intersecting constraints through systematic information collection ³¹. The agent should prioritize tasks that maximize the mutual information $I(Q;Y)$ gained from each question Q about the target variable Y ³¹.

In practice, this means the agent should assess the current state of its knowledge base and identify areas of high uncertainty. It should then generate questions that, if answered, would most significantly reduce that uncertainty. For instance, if the agent has high-resolution SAR data but lacks ground-truth soil moisture measurements, the uncertainty in its soil moisture model is high. Fetching ground-truth data would provide high mutual information, significantly reducing uncertainty and improving model accuracy. Therefore, the agent should prioritize fetching ground-truth data over other tasks.

3.3 Urgency, Importance, and Easiness

A rational model of task priority suggests assessing each task based on Urgency, Importance, and Easiness subjectively, in isolation ³⁸. - **Urgency:** How time-sensitive is the data? For example, real-time weather data for hive health monitoring is highly urgent, while historical climate data is less so. - **Importance:** How critical is the task to the overall mission? Mapping Drone Congregation Areas is highly important for queen rearing, while optimizing code comments is less important. - **Easiness:** How computationally expensive or complex is the task? Simple statistical analyses are easier than complex machine learning model training.

The agent should use a weighted scoring system to prioritize tasks based on these three dimensions. High-urgency, high-importance, and low-easiness tasks should be prioritized first, as they offer the greatest value for the effort invested. This ensures that the agent focuses its resources on the most impactful scientific inquiries.

3.4 Verifiability and Objective Success Signals

Self-improvement only works reliably in domains where outcomes are objectively verifiable, such as code compilation or math proofs ². In domains without a clear signal for 'better,' systems tend to hack reward functions or oscillate without meaningful progress ². Therefore, WENDY should prioritize scientific domains where binary success signals are available to guide reinforcement learning loops ³².

For ecological research, verifiability can be achieved through: - **Statistical Significance:** Does the new model significantly improve upon the baseline? - **Cross-Validation:** Does the model perform well on unseen data? - **Ground-Truth Comparison:** Do the model predictions match real-world measurements?

By prioritizing tasks with clear verifiability metrics, the agent can ensure that its self-improvement efforts are directed toward genuine scientific advancement rather than superficial optimizations. For example, when developing a soil moisture model, the agent should prioritize tasks that improve the R-squared value against ground-truth data, as this is an objective measure of model accuracy ¹⁶.

3.5 Data Sparsity and Anomaly Detection

Current research shows that a primary bottleneck in elevating agents to higher autonomy levels is the inability to proactively identify valuable tasks ⁴. Key components for enhancing agentic architecture include Perception, which involves environmental understanding and anomaly detection ⁴.

WENDY should prioritize domains where data sparsity or anomalies are detected. Sparse data indicates a lack of knowledge, while anomalies may indicate novel phenomena or errors in data collection. By focusing on these areas, the agent can maximize the value of its research efforts. For instance, if the agent detects a sudden drop in SAR backscatter coefficients in a specific region, it should prioritize investigating this anomaly, as it may indicate a significant ecological event, such as a forest fire or construction disturbance.

Chapter 4: Protocols for External Data Fetching vs. Local Analysis

4.1 Layered Context Management Architecture

The decision to fetch external open-science data versus analyzing local binary files must be governed by a layered context management architecture. Static context should be reserved for universal, stable data like compliance rules, while dynamic context should be fetched at runtime based on specific query requirements ²⁹.

A practical framework for deciding between static and dynamic context involves four questions: 1. **How often is this needed?** Universal needs suggest static context; sometimes/dependent needs suggest dynamic context. 2. **How large is this information?** Small information ($< \sim 500$ tokens) suggests static context; large information suggests dynamic context. 3. **How often does this change?** Stable information suggests static context; frequently changing information suggests dynamic context. 4. **What happens if it's missing?** Critical information suggests static context; supplemental information suggests dynamic context ²⁹.

For WENDY, local binary files in `/data/` represent static or session-specific context that is readily available and low-latency. External open-science repositories represent dynamic context that is large, variable, and high-latency. The agent should prioritize local analysis when the local data is sufficient to answer the current question with high confidence. It should only fetch external data when the local data is sparse, outdated, or insufficient to reduce uncertainty to an acceptable level.

4.2 Retrieval-Augmented Generation for Grounding

WENDY should utilize Retrieval-Augmented Generation (RAG) to ground her plans in external knowledge sources like NASA Earthdata or ESA Copernicus when local data lacks sufficient temporal or spatial resolution ³. RAG allows the agent to access up-to-date, authoritative scientific data without having to store it locally, ensuring that its analyses are based on the best available information.

However, fetching external data introduces latency and potential hallucination risks. To mitigate this, WENDY must employ strong reflection mechanisms that ground critiques in external evidence, such as re-fetching data from databases to verify numerical claims, rather than relying on internal consistency checks which are vulnerable to consistent hallucination ²¹.

The Model Context Protocol (MCP) offers a secure, decoupled architecture for connecting to external resources without exposing sensitive data, ensuring that tool invocation remains separate from LLM responses ¹. MCP uses a client-server architecture to decouple tool invocation from LLM responses, enhancing security and interoperability ¹. By using MCP, WENDY can safely access external datasets without compromising the integrity of its local environment.

4.3 Strong Reflection and External Verification

A key distinction is made between 'weak' and 'strong' reflection. Weak reflection involves a single LLM call critiquing its own draft internally, which is vulnerable to 'consistent hallucination'—errors that are internally coherent with the model's world-model. Strong reflection grounds the critique in external evidence, such as SQL re-fetches or retrieved documents, enabling it to detect and correct factual errors ²¹.

The critique phase of the reflection architecture is designed with opposing objectives to the generation phase. The generator is instructed to be thorough, while the critic is instructed to be sceptical, actively trying to find faults rather than validate the output ²¹. This multi-agent design, where the critic has access to external data the generator never saw, is the structural reason this approach catches errors internal critique cannot ²¹.

For WENDY, this means that after generating a preliminary scientific report based on local data, a Critic Agent should verify the findings against external sources. If the local data suggests a certain soil moisture level, the Critic should fetch recent satellite data or ground-truth measurements to confirm or refute this finding. Only after this external verification should the final report be generated.

4.4 Minimizing Computational Dissipation

If local binary files in `/data/` provide robust sampling with low standard deviation, the agent should prioritize local analysis to minimize computational dissipation and network overhead ³. Fetching external data is computationally expensive and time-consuming. Therefore, the agent should only do so when necessary.

The agent should assess the quality of the local data before deciding to fetch external data. Metrics such as data completeness, temporal resolution, and spatial coverage should be used to determine if the local data is sufficient. If the local data is high-quality and relevant, the agent should proceed with local analysis. If the local data is sparse or outdated, the agent should fetch external data to supplement it.

Chapter 5: Dynamic Selection of Output Formats

5.1 Token Efficiency and Parsing Convenience

The selection between Markdown and Jupyter Notebooks must be driven by token efficiency and the intended consumer of the output. Research indicates that editing Jupyter notebooks requires significantly more resources—1.94x total tokens and 2.53x fresh input tokens—compared to plain text Python files, primarily due to the overhead of managing JSON structure and extracting stored outputs ¹¹.

Therefore, for internal agent-to-agent communication or rapid iterative debugging, Markdown or plain text formats are superior due to their lower token cost and parsing convenience ^{9 12}. Markdown costs 3x fewer tokens than HTML, making it a more efficient format for AI agents ⁹. Additionally, studies comparing JSON, XML, and Markdown for AI agents found that both JSON and Markdown performed nearly identically overall, with the choice depending on parsing convenience within the existing technology stack ¹².

For WENDY, this means that intermediate reports, status updates, and internal communications should be generated in Markdown. This minimizes token usage and speeds up the processing cycle, allowing the agent to iterate more quickly.

5.2 Reproducibility and Visual Output

However, for final scientific reporting and human review, Jupyter Notebooks remain essential for combining executable code, narrative text, and visualizations in a reproducible format ^{13 14}. Jupyter Notebooks allow researchers to see not just the final results, but also the code and logic used to generate them, ensuring transparency and reproducibility.

WENDY should dynamically select the output format based on the complexity of the analysis: using Markdown for lightweight status updates and structural planning, and Jupyter Notebooks for complex statistical analyses requiring executable plotting commands and spatial map rendering ¹⁰.

The DatawiseAgent framework, a notebook-centric LLM agent, uses a Finite State Transducer (FST)-based multi-stage design with a unified interaction representation that combines markdown cells for reasoning and documentation with executable code cells for logic ¹⁰. This approach allows the agent to progressively build a solution while

leveraging real-time feedback from the execution environment. WENDY can adopt a similar approach, generating Jupyter Notebooks for complex tasks that require iterative refinement and visual output.

5.3 Quarto for Multi-Format Publishing

The agent should also consider Quarto as a modern, language-agnostic publishing system for larger-scale workflows that require multiple output formats ¹³. Quarto supports multiple programming languages including R, Python, Julia, and Observable, and has a cleaner structure for larger-scale publishing workflows ¹³.

For WENDY, Quarto could be used to generate comprehensive scientific reports that include both Markdown text and executable code chunks, which can then be rendered into HTML, PDF, or Word formats for distribution. This provides flexibility in how the reports are consumed, allowing them to be shared with different stakeholders in their preferred format.

5.4 Dynamic Format Selection Logic

The dynamic selection of output formats should be governed by a simple set of rules: 1. **Internal Communication:** Use Markdown for low-token, high-speed internal updates. 2. **Simple Analysis:** Use Markdown for reports that do not require executable code or complex visualizations. 3. **Complex Analysis:** Use Jupyter Notebooks for reports that require executable code, statistical plots, or spatial maps. 4. **Multi-Format Distribution:** Use Quarto for reports that need to be distributed in multiple formats (HTML, PDF, Word).

The agent should assess the complexity of the task and the intended audience before selecting the output format. This ensures that the output is both efficient and appropriate for its intended use.

Chapter 6: Implementing the Feedback Loop for Cognitive Independence

6.1 The Self-Improvement Cycle

To close the loop and achieve true cognitive independence, WENDY must implement a continuous self-improvement cycle consisting of an Executor Agent, an Evaluator Agent, a Prompt Optimizer, and a Memory System ³⁵. - **Executor Agent:** Interacts with the environment to generate responses and execute tasks. - **Evaluator Agent:** Analyzes the executor's output, assigns a score, and identifies weaknesses. - **Prompt Optimizer:** Uses evaluator feedback to refine instructions for future executions. - **Memory System:** Stores past interactions, responses, and feedback in a vector database to allow retrieval and application of learned improvements ³⁵.

This workflow creates a closed-loop process: User Task → Executor generates response → Evaluator scores the response → Prompt Optimizer improves instructions → Memory system stores learning → Next iteration becomes smarter ³⁵. Over multiple iterations, this leads to higher quality results without manual prompt rewriting.

6.2 Hierarchical Memory Architectures

Scientific agents employ several distinct strategies for integrating with external tools, representing a spectrum of autonomy ³. An LLM-based scientific agent's memory mechanism is fundamental for sustained research and cumulative learning. It acts as both working memory for immediate tasks and a persistent knowledge base, archiving all past outcomes ³. By retaining successful protocols and failed inquiries, this long-term memory enables the agent to learn from experience, refine strategies, avoid repeating errors, and systematically drive the discovery process forward ³.

Sophisticated memory architectures are critical for enabling agents to maintain context and learn from interactions over extended periods. These systems often implement hierarchical structures, including short-term context buffers for immediate interactions and long-term memory for preference learning and consistency maintenance across sessions ¹⁹.

The Auton Agentic AI Framework implements a hierarchical memory architecture inspired by biological episodic memory. Short-term memory is a high-fidelity event stream. Long-term memory consists of three stores: Semantic Memory (stable facts),

Episodic Memory (compressed records of past interactions), and Procedural Memory (reusable plans) ²⁸. A 'Reflector-Driven Consolidation Protocol' transfers information from short-term to long-term memory. When a session ends, a background 'Reflector Agent' processes the event stream by segmenting it into episodes, extracting salient insights, vectorizing them for semantic retrieval, and storing them in the knowledge base ²⁸.

For WENDY, this means implementing a memory system that stores not just the final reports, but also the reasoning traces, data sources, and evaluation scores for each task. This allows the agent to learn from its past experiences and improve its future performance.

6.3 The ReAct and CodeAct Frameworks

The ReAct (Reasoning and Acting) framework is a key mechanism for reflective and iterative tool use. It operates in a continuous Thought-Action-Observation loop, allowing an agent to reason about a problem, call a tool, observe the output, reflect on the result, and then decide on the next logical step ³ ²³. This process mimics human problem-solving and enables adaptation based on feedback.

Reflective and Iterative Tool Use can be extended into the 'CodeAct' framework, which specializes in generating and executing code within a sandboxed environment. After execution, the agent observes the output or errors, reflects on them, iteratively corrects the code, and retries until the task is completed successfully ²³.

For WENDY, this means that when generating a scientific report, the agent should not just write text, but also generate and execute Python code to perform the necessary statistical analyses. If the code fails, the agent should reflect on the error, correct the code, and retry. This ensures that the final report is based on accurate, verified computations.

6.4 Multi-Agent Critique and Red Teaming

The multi-agent critique pattern, also known as Red Teaming, enhances robustness by separating roles. A 'Generator Agent' creates a plan, while a 'Critic Agent' explicitly reviews it for errors ²². This separation mitigates confirmation bias and significantly increases the agent's reliability ²².

The core problem with standard one-shot generation is that the model has no opportunity to review its own output. Errors introduced during generation are delivered directly to the user without any internal check ²¹. A multi-phase architecture becomes necessary over simpler approaches when the critique phase needs to perform actions outside the token stream, such as re-fetching data from a database to verify numerical claims ²¹.

For WENDY, this means implementing a multi-agent system where one agent generates the scientific report, and another agent critiques it. The Critic Agent should have access to external data sources and verification tools, allowing it to ground its critique in factual evidence. This ensures that the final report is accurate, reliable, and free from hallucinations.

Chapter 7: Synthesis and Architectural Recommendations

7.1 Transitioning from Static to Dynamic Autonomy

The transition from a static JSON instruction queue to a dynamic, self-questioning architecture requires a fundamental shift in how WENDY perceives and interacts with its environment. Instead of waiting for instructions, WENDY must actively scan its data landscape, identify knowledge gaps, and generate its own research questions. This requires the implementation of the Q-Centric Model of Intelligence, with its components of epistemic instability detection, question formation, and experience-looped evaluation ³³.

The SQLM framework provides a practical mechanism for implementing this self-questioning capability, using asymmetric self-play to generate and solve increasingly complex scientific problems ³⁴. By rewarding the proposer for generating high-value questions and the solver for providing accurate answers, the system can autonomously improve its scientific reasoning capabilities.

7.2 Prioritization Through Value of Information

To ensure that WENDY focuses its resources on the most impactful research, it must use Value of Information analysis to prioritize tasks ³⁶. By calculating the mutual

information gained from potential questions, the agent can identify areas of high uncertainty and prioritize tasks that reduce that uncertainty ³¹.

This prioritization should be guided by metrics of urgency, importance, and easiness, ensuring that high-value, low-effort tasks are executed first ³⁸. Additionally, the agent should prioritize domains where outcomes are objectively verifiable, allowing it to use binary success signals to guide its self-improvement loops ².

7.3 Hybrid Data Strategy

WENDY should adopt a hybrid data strategy, prioritizing local analysis when local data is sufficient and fetching external data only when necessary to reduce uncertainty ²⁹. This minimizes computational dissipation and network overhead while ensuring that the agent has access to the best available information ³.

The use of Retrieval-Augmented Generation (RAG) and the Model Context Protocol (MCP) allows the agent to safely and efficiently access external data sources, grounding its analyses in authoritative scientific data ¹ ³. Strong reflection mechanisms, grounded in external evidence, ensure that the agent's outputs are accurate and free from hallucinations ²¹.

7.4 Dynamic Output Formatting

The selection of output formats should be driven by token efficiency and the intended consumer of the output. Markdown should be used for internal communications and simple reports, while Jupyter Notebooks should be used for complex analyses requiring executable code and visualizations ¹¹ ¹⁴.

Quarto can be used for multi-format publishing, allowing WENDY to distribute its reports in a variety of formats suitable for different stakeholders ¹³. This dynamic formatting ensures that the agent's outputs are both efficient and appropriate for their intended use.

7.5 Continuous Self-Improvement

Finally, WENDY must implement a continuous self-improvement cycle, consisting of an Executor, Evaluator, Prompt Optimizer, and Memory System ³⁵. This closed-loop process allows the agent to learn from its past experiences and improve its future performance.

Hierarchical memory architectures, such as those implemented in the Auton Agentic AI Framework, allow the agent to store and retrieve past experiences, enabling cumulative learning and strategic refinement ²⁸. The ReAct and CodeAct frameworks provide mechanisms for iterative refinement and error correction, ensuring that the agent's outputs are accurate and reliable ²³. Multi-agent critique patterns, such as Red Teaming, further enhance robustness by separating generation and evaluation roles, mitigating confirmation bias and improving overall reliability ²².

Conclusion

The architectural limitations of WENDY's current static JSON instruction queue system prevent it from achieving true cognitive independence. By relying on human-defined tasks, the system is confined to low levels of autonomy, lacking the intrinsic motivation and strategic reasoning required for high-level scientific discovery.

To overcome these limitations, WENDY must adopt a new autonomous self-questioning mechanism based on the Q-Centric Model of Intelligence. This mechanism, supported by asymmetric self-play frameworks, Value of Information analysis, and hierarchical memory architectures, enables the agent to independently generate, prioritize, and execute complex scientific research tasks.

By implementing a hybrid data strategy, dynamic output formatting, and a continuous self-improvement cycle, WENDY can evolve into a self-evolving scientific institution, capable of addressing critical real-world challenges with the rigor and accuracy demanded by space agencies and ecological research institutions. This transformation moves the system beyond procedural execution to strategic reasoning, allowing it to identify its own knowledge gaps, create the necessary instructions to fill them, and operate with true cognitive independence.

The path forward is clear: WENDY must stop waiting for instructions and start asking questions. By doing so, it will not only improve its own capabilities but also contribute to the broader scientific community, generating substantial, verifiable, and impactful research that advances our understanding of the natural world.

Reference

1. A Survey of AI Agent Protocols <https://arxiv.org/html/2504.16736v2>
2. Self-Improving AI Agents: The 2026 Guide | Articles | o-mega <https://o-mega.ai/articles/self-improving-ai-agents-the-2026-guide>
3. Autonomous Agents for Scientific Discovery <https://arxiv.org/html/2510.09901v1>
4. HKUSTDial/awesome-data-agents <https://github.com/hkustdial/awesome-data-agents>
5. Can AI Conduct Autonomous Scientific Research? Case ... <https://www.biorxiv.org/content/10.64898/2026.01.05.697809v1.full>
6. From Models to Scientists: Building AI Agents for Scientific ... <https://kempnerinstitute.harvard.edu/research/deeper-learning/from-models-to-scientists-building-ai-agents-for-scientific-discovery/>
7. Building An Autonomous and Open Data Scientist Agent ... <https://www.together.ai/blog/building-an-autonomous-and-open-data-scientist-agent-from-scratch>
8. OpenScience.ai — Autonomous AI Agents Conducting ... <https://openscience.ai/>
9. HTML vs Markdown for AI Agents: Which Format Wins in ... <https://beam.ai/agentics-insights/html-vs-markdown-which-format-actually-makes-ai-agents-more-useful>
10. DatawiseAgent: A Notebook-Centric LLM Agent ... <https://arxiv.org/html/2503.07044v1>
11. How File Format Shapes Agent Work <https://ai.colab.duke.edu/colab-ai-blog/all-blogs/how-file-format-shapes-agent-work-an-experiment-with-jupyter-notebooks/>
12. Does Output Format Actually Matter? An Experiment ... <https://checksum.ai/blog/does-output-format-actually-matter-an-experiment-comparing-json-xml-and-markdown-for-llm-tasks>
13. R Markdown vs. Quarto for automated reports <https://community.appliedepi.org/t/r-markdown-vs-quarto-for-automated-reports-which-one-do-you-prefer/8223>
14. 27 R Markdown - R for Data Science - Hadley Wickham <https://r4ds.had.co.nz/r-markdown.html>
15. Soil moisture estimates at 1 km resolution making a ... - HESS <https://hess.copernicus.org/articles/27/1221/2023/>
16. Soil Moisture Mapping Using Sentinel-1 SAR Data and ... <https://www.mdpi.com/3042-5743/36/1/9>

17. JSON Prompting: The Hidden Architecture of Deterministic ... <https://pub.towardsai.net/json-prompting-the-hidden-architecture-of-deterministic-ai-interaction-622341d87ff3>
18. Designing AI APIs with JSON-Safe Outputs - Level Up Coding <https://levelup.gitconnected.com/designing-ai-apis-with-json-safe-outputs-0c49c6e39a73>
19. Distinguishing Autonomous AI Agents from Collaborative ... <https://arxiv.org/html/2506.01438v1>
20. Task Decomposition by Autonomous AI Agents <https://www.linkedin.com/pulse/task-decomposition-autonomous-ai-agents-principles-andre-9nmee>
21. Reflection Agent Architecture: Eliminating LLM ... <https://pub.towardsai.net/reflection-agent-architecture-eliminating-llm-hallucinations-via-tool-grounded-iterative-9a61767cc979>
22. AI-Agents 03 - Self-Reflection in Agentic Systems <https://mschoeffel.de/en/blog/ai-agents-03-self-reflection>
23. Autonomous Agents for Scientific Discovery: Orchestrating ... <https://arxiv.org/html/2510.09901v2>
24. Monitor, troubleshoot, and improve AI agents with Datadog <https://www.datadoghq.com/blog/monitor-ai-agents/>
25. Handling AI Agent Health Check Failures <https://synergetics.ai/blog/handling-ai-agent-health-check-failures>
26. Agent Observability: How to Monitor AI Agents <https://www.rubrik.com/insights/ai-observability>
27. Empowering Autonomous AI Agents through Dynamic Tool ... <https://dev.to/gde/empowering-autonomous-ai-agents-through-dynamic-tool-creation-3pfm>
28. The Auton Agentic AI Framework A Declarative ... <https://arxiv.org/html/2602.23720v1>
29. Static Context vs Dynamic Context in AI Agents <https://www.mindstudio.ai/blog/static-context-vs-dynamic-context-ai-agents>
30. Empowering Autonomous AI Agents through Dynamic Tool ... <https://gist.github.com/tanaichek/5c97352f92a6203577efb076e0a7209f>
31. First Ask Then Answer: A Framework Design for AI ... <https://arxiv.org/html/2508.08308v1>
32. Self-Questioning Techniques for Large Language Models <https://www.linkedin.com/top-content/technology/ai-language-processing/self-questioning-techniques-for-large-language-models/>
33. From answers to questions: the Q-centric model of intelligence <https://link.springer.com/article/10.1007/s44163-025-00402-w>

34. Self-Questioning Language Models <https://www.alphaxiv.org/abs/2508.03682>
35. The Architecture Behind Self-Improving AI Agents (Full ... <https://medium.com/@rselvaguru/the-architecture-behind-self-improving-ai-agents-full-implementation-guide-dfb0ce67c7c7>
36. ISPOR Report Value of Information Analysis for Research ... <https://www.sciencedirect.com/science/article/pii/S1098301520300279>
37. A Tool to Improve Research Prioritization and Reduce Waste https://www.researchgate.net/publication/282401995_Value_of_Information_A_Tool_to_Improve_Research_Prioritization_and_Reduce_Waste
38. A Rational Model of Task Priority | by Tom Gidden <https://medium.com/@gid/a-rational-model-of-task-priority-16789f50287d>
39. A framework for better sensor-based beehive health ... <https://www.sciencedirect.com/science/article/pii/S0168169923002946>
40. Sound monitoring assesses bee hive health with 92% ... <https://www.facebook.com/groups/treatmentfreebeekeepers/posts/5663684850313995/>